



UNIVERSIDADE FEDERAL DO AMAZONAS
FACULDADE DE TECNOLOGIA
SISTEMAS OPERACIONAIS

SISTEMAS OPERACIONAIS

Trabalho 01

Manaus-AM

2026

Alexandre Antonaccio Senna
Fabricio Lessa Lorenzi Filho

Trabalho 01

Sistemas Operacionais

Relatório sobre o desenvolvimento do primeiro
trabalho da matéria de Sistemas Operacionais

Manaus-AM
2026

RELATÓRIO DE IMPLEMENTAÇÃO - PROBLEMAS CLÁSSICOS DE CONCURRENCIA

Resumo

Este documento apresenta comentários gerais sobre as escolhas realizadas pela equipe durante a implementação dos problemas clássicos de concorrência em Sistemas Operacionais. Foram desenvolvidas versões com controle de concorrência e versões sem controle, permitindo observar os impactos das condições de corrida (race conditions) em ambientes concorrentes.

Sumário

1. Bibliotecas Utilizadas	4
2. Problema Produtor/Consumidor	4
3. Versões Sem Controle de Concorrência	4
4. Problema Leitores/Escritores	5
5. Ambiente de Desenvolvimento	5
6. Processos Leitores x Escritores	5
6.1 Temática	5
6.2 Leitura Suja	5
6.3 Escritores com prioridade	5
6.4 Sem Controle	6
7. Processos Produtores X Consumidores	7
7.1 Múltiplos Produtores, consumidor único	7
7.2 Múltiplos Produtores e Consumidores	8
7.3 Sem Controle	9
8. Conclusão	10
9. Referências e Fontes	10

1. Bibliotecas Utilizadas

O trabalho foi desenvolvido em linguagem C utilizando a biblioteca POSIX Threads (pthread) juntamente com semáforos (semaphore.h). As threads foram utilizadas para representar produtores, consumidores, leitores e escritores executando simultaneamente.

Para garantir sincronização e exclusão mútua nas versões corretas, foram utilizados:

Tabela 1: Mecanismos de Sincronização Utilizados no Projeto

Mecanismo	Função
pthread_mutex_t	Garantir exclusão mútua em regiões críticas
sem_t empty	Controlar vagas livres no buffer
sem_t full	Controlar quantidade de itens disponíveis
Semáforo recurso	Controlar acesso ao recurso compartilhado

2. Problema Produtor/Consumidor

Nos programas Produtor/Consumidor, foi utilizado um buffer circular compartilhado entre múltiplas threads produtoras e consumidoras. O uso de semáforos permitiu controlar corretamente a ocupação do buffer, enquanto o mutex garantiu acesso exclusivo durante modificações.

```
sem_wait(&empty);  
pthread_mutex_lock(&mutex_buffer);  
  
/* Região crítica */  
  
pthread_mutex_unlock(&mutex_buffer);  
sem_post(&full);
```

Durante os testes, observou-se que:

- não ocorreram sobrescritas de dados;
- o buffer manteve consistência;
- produtores aguardaram corretamente quando o buffer ficou cheio;
- consumidores aguardaram corretamente quando o buffer ficou vazio.

3. Versões Sem Controle de Concorrência

Nas versões sem sincronização, não foram utilizados mutexes nem semáforos. Isso permitiu demonstrar problemas clássicos de concorrência, como:

- 1.0. Race conditions;
- 1.1. sobrescrita de posições do buffer;
- 1.2. leitura de valores inválidos;
- 1.3. corrupção dos dados compartilhados;
- 1.4. inconsistência no saldo das contas.

Os resultados mostraram claramente a importância dos mecanismos de sincronização em sistemas concorrentes. Em diversos momentos, consumidores leram posições vazias do buffer e escritores modificaram simultaneamente os mesmos dados, causando corrupção do estado global do sistema.

4. Problema Leitores/Escritores

Foi implementada a solução clássica de prioridade para escritores. Nessa abordagem, novos leitores são bloqueados caso exista algum escritor aguardando acesso ao recurso compartilhado.

Os testes mostraram que múltiplos leitores conseguem acessar simultaneamente os dados quando não há escritores ativos, enquanto escritores possuem acesso exclusivo à região crítica.

5. Ambiente de Desenvolvimento

O projeto foi executado utilizando GCC em ambiente Linux com suporte a pthreads. Também foram utilizados arquivos de configuração do Visual Studio Code para facilitar compilação, depuração e execução dos programas.

6. Processos Leitores x Escritores

6.1 Temática

Foi optado por um sistema bancário onde são realizadas transferências bancárias entre duas contas, desta forma foi possível visualizar claramente os eventos de estudo do trabalho

6.2 Leitura Suja

```
Sistema Bancario Iniciado. Saldo Total Inicial: 1000

[Escritor 1] Criado e tentando entrar na RC...
[Escritor 1] Entrou na RC. Iniciando transferencia de 73 da Conta A para B.
[Escritor 2] Criado e tentando entrar na RC...
[Leitor 1] Lendo dados (SEM BLOQUEIO)...

---> [LEITURA SUJA - Leitor 1] Total inconsistente! Total lido: 927
```

Figura 1: Exemplo de Leitura Suja no sistema bancário.

O código é estruturado de forma que ao realizar uma transferência de uma conta A, para uma conta B existe um tempo significativo, tal que a thread na qual o leitor é executado é lido que o somatório dos saldos é inferior a 1000, isso ocorre por que é lido o novo valor da conta A, e o valor anterior a transferência da conta B.

```
pthread_mutex_lock(&mutex_escrita); // Entra na Região Crítica
printf("[Escritor %d] Entrou na RC. Iniciando transferencia de %d da Conta A para B.\n", id, valor);

// operacao demorada para forcar a leitura suja
contaA -= valor;
usleep(100000);
contaB += valor;

printf("[Escritor %d] Transferencia concluida. (ContaA: %d, ContaB: %d, Total: %d). Saindo da RC.\n",
       id, contaA, contaB, contaA + contaB);
pthread_mutex_unlock(&mutex_escrita); // Sai da Região Crítica
```

Figura 2: Escritor para o exemplo de leitura suja

6.3 Escritores com prioridade

Quando escritores entram em região crítica (RC) novos leitores são bloqueados até que o processo de transferência seja completo, desta forma é possível ver que os leitores são bloqueados durante o processo,

evitando a leitura suja, evidenciado pelo fato de que o somatório dos saldos sempre é igual a 1000 dada a finalização da transferência.

```
// Protocolo de Entrada - Escritor
pthread_mutex_lock(&wmutex);
write_count++;
if (write_count == 1) sem_wait(&read_try); // Bloqueia novos leitores
pthread_mutex_unlock(&wmutex);

sem_wait(&recurso); // Entra na RC

printf("[Escritor %d] Entrou na RC. Bloqueou leitores. Transferindo %d...\n", id, valor);
contaA -= valor;
usleep(100000); // Simulando lentidão
contaB += valor;
printf("[Escritor %d] Transferencia finalizada. (Total: %d). Saindo da RC.\n", id, contaA + contaB);

sem_post(&recurso); // Sai da RC

// Protocolo de Saída - Escritor
pthread_mutex_lock(&wmutex);
write_count--;
if (write_count == 0) sem_post(&read_try); // Libera leitores se não houver mais escritores
pthread_mutex_unlock(&wmutex);

sleep(1);
```

Figura 3: Correção da leitura suja com prioridade para escritores.

```
Sistema Bancario Iniciado. Saldo Total Inicial: 1000.

[Escritor 1] Criado. Tentando obter prioridade...
[Escritor 1] Entrou na RC. Bloqueou leitores. Transferindo 84...
[Leitor 1] Tentando ler...
[Escritor 2] Criado. Tentando obter prioridade...
[Leitor 2] Tentando ler...
[Leitor 3] Tentando ler...
[Leitor 4] Tentando ler...
[Leitor 5] Tentando ler...
[Escritor 1] Transferencia finalizada. (Total: 1000). Saindo da RC.
[Escritor 2] Entrou na RC. Bloqueou leitores. Transferindo 87...
[Escritor 2] Transferencia finalizada. (Total: 1000). Saindo da RC.
[Leitor 1] Lendo na RC. Total na conta: 1000
```

Figura 4: Fluxo de execução sincronizado.

6.4 Sem Controle

Quando não é realizado o controle de prioridade dos processos, ocorrem escritas em momentos errados (propositalmente utilizando sleep em escritores), de forma que o saldo final é inconsistente com as transferências bancárias realizadas.

```

[Escritor 3] Terminou de escrever.
[Escritor 2] Terminou de escrever.
[Leitor 1] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 2] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 5] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 4] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 3] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 1] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 2] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 5] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 4] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 3] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 2] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 1] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 4] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 3] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895
[Leitor 5] Leu saldo -> ContaA: 329 | ContaB: 566 | Total: 895

Resultado Final (Deveria ser 1000): 895

```

Figura 5: Resultados sem controle de concorrência.

```

printf("[Escritor %d] Transferindo %d (SEM RC)...\n", id, valor);

int tempA = contaA;
usleep(rand() % 50000); // Dorme para aumentar chance de colisão
contaA = tempA - valor;
int tempB = contaB;
usleep(rand() % 50000);
contaB = tempB + valor;

printf("[Escritor %d] Terminou de escrever.\n", id);

```

Figura 6: Inserção de tempo artificialmente entre operações.

7. Processos Produtores X Consumidores

7.1 Múltiplos Produtores, consumidor único

É feito o controle de fluxo de forma que produtores sempre inserem no buffer em posição vazia ou esperam o consumo de alguma posição de forma que o buffer não é corrompido e nenhuma posição é sobrescrita

```

printf("[Produtor %d] Produziu o item %d. Dormindo/Aguardando vaga...\n", id, item);

sem_wait(&empty); // Se buffer cheio, bloqueia aqui ("dorme")
pthread_mutex_lock(&mutex_buffer); // Entra RC

printf("[Produtor %d] Entrou na RC.\n", id);
buffer[in] = item;
in = (in + 1) % TAM_BUFFER;
imprimir_buffer();
printf("[Produtor %d] Saiu da RC.\n", id);

pthread_mutex_unlock(&mutex_buffer); // Sai RC
sem_post(&full); // Avisar que tem novo item

usleep((rand() % 500) * 1000); // Atraso aleatório

```

Figura 7: Execução com múltiplos produtores e consumidor único.

```

[Consumidor 1] Aguardando item no buffer (Dormindo se vazio)...
[Produtor 1] Produziu o item 100. Dormindo/Aguardando vaga...
[Produtor 1] Entrou na RC.
BUFFER: [ 100 [Produtor 2] Produziu o item 200. Dormindo/Aguardando vaga...
- - - - ]
[Produtor 1] Saiu da RC.
[Produtor 3] Produziu o item 300. Dormindo/Aguardando vaga...
[Produtor 3] Entrou na RC.
BUFFER: [ 100 300 - - - ]
[Produtor 3] Saiu da RC.
[Produtor 2] Entrou na RC.
BUFFER: [ 100 300 200 - - ]
[Produtor 2] Saiu da RC.
[Consumidor 1] Entrou na RC. Consumiu o item 100.

```

Figura 8: Detalhe da sincronização do buffer.

```

BUFFER: [ 301 300 200 201 101 ]
[Produtor 3] Saiu da RC.
[Produtor 1] Produziu o item 102. Dormindo/Aguardando vaga...
[Produtor 2] Produziu o item 202. Dormindo/Aguardando vaga...
[Produtor 3] Produziu o item 302. Dormindo/Aguardando vaga...
[Consumidor 1] Aguardando item no buffer (Dormindo se vazio)...
[Consumidor 1] Entrou na RC. Consumiu o item 300.
BUFFER: [ 301 - 200 201 101 ]
[Consumidor 1] Saiu da RC.
[Produtor 1] Entrou na RC.
BUFFER: [ 301 102 200 201 101 ]
[Produtor 1] Saiu da RC.
[Produtor 1] Produziu o item 103. Dormindo/Aguardando vaga...
[Consumidor 1] Aguardando item no buffer (Dormindo se vazio)...
[Consumidor 1] Entrou na RC. Consumiu o item 200.
BUFFER: [ 301 102 - 201 101 ]
[Consumidor 1] Saiu da RC.
[Produtor 2] Entrou na RC.
BUFFER: [ 301 102 202 201 101 ]

```

Figura 9: Gráfico de ocupação do buffer.

7.2 Múltiplos Produtores e Consumidores

Problema similar ao anterior onde múltiplos consumidores trabalham juntos no mesmo buffer, o controle de fluxo funciona de forma similar


```

sem_wait(&full);
pthread_mutex_lock(&mutex_buffer);

if (itens_consumidos >= total_gerado) {
    // Se já consumiu tudo, sai e libera para outros sairem
    pthread_mutex_unlock(&mutex_buffer);
    sem_post(&full);
    break;
}

int item = buffer[out];
buffer[out] = 0;
out = (out + 1) % TAM_BUFFER;
itens_consumidos++;

printf("[Consumidor %d] Status: CONSUMINDO item %d.\n", id, item);
imprimir_buffer();

pthread_mutex_unlock(&mutex_buffer);
sem_post(&empty);

sleep(1); // Consumidor mais lento

```

Figura 10: Fluxo de consumidores com múltiplos produtores e consumidores.

7.3 Sem Controle

Sem a utilização de controle é possível observar consumo em posições vazias e perda de dados, foram inseridos intervalos de tempo de modo a forçar os erros observados.

```

printf("[Produtor %d] Produzindo %d (Sem Semáforo/Mutex)\n", id, item);

// Colocando no buffer sem proteção. Ocorre sobrescrita!
buffer[in] = item;
usleep(50000); // Pausa no meio da operação para forçar erro de in
in = (in + 1) % TAM_BUFFER;

imprimir_buffer();
sleep(1);

```

Figura 11: Inserção artificial de tempo em produtor.

```

// Consumindo sem verificar se está vazio.
int temp_out = out;
usleep(20000); // Pausa para forçar conflito no ponteiro out
int item = buffer[temp_out];
buffer[temp_out] = 0; // Limpa slot
out = (temp_out + 1) % TAM_BUFFER;

printf("----> [ERRO/Consumo] [Consumidor %d] Consumiu %d na posicao %d\n", id, item, temp_out);

sleep(1);

```

Figura 12: Inserção artificial de tempo em consumidor.

```

----> [ERRO/Consumo] [Consumidor 1] Consumiu 0 na posicao 1
----> [ERRO/Consumo] [Consumidor 2] Consumiu 0 na posicao 1
[Produtor 2] Produzindo 21 (Sem Semáforo/Mutex)
[Produtor 1] Produzindo 11 (Sem Semáforo/Mutex)
BUFFER: [ 0 0 11 0 0 ]
BUFFER: [ 0 0 11 0 0 ]
----> [ERRO/Consumo] [Consumidor 1] Consumiu 11 na posicao 2
----> [ERRO/Consumo] [Consumidor 2] Consumiu 0 na posicao 2
[Produtor 1] Produzindo 12 (Sem Semáforo/Mutex)
[Produtor 2] Produzindo 22 (Sem Semáforo/Mutex)
BUFFER: [ BUFFER: [ 0 0 0 0 22 ]
0 0 0 0 22 ]
----> [ERRO/Consumo] [Consumidor 1] Consumiu 0 na posicao 3
----> [ERRO/Consumo] [Consumidor 2] Consumiu 0 na posicao 3
[Produtor 2] Produzindo 23 (Sem Semáforo/Mutex)
[Produtor 1] Produzindo 13 (Sem Semáforo/Mutex)
BUFFER: [ 13 0 0 0 22 ]
BUFFER: [ 13 0 0 0 22 ]

```

Figura 13: Resumo dos impactos da falta de sincronização.

8. Conclusão

A implementação permitiu compreender de forma prática os principais problemas de concorrência estudados em Sistemas Operacionais. As versões sincronizadas demonstraram funcionamento correto e seguro, enquanto as versões sem controle evidenciaram os efeitos das condições de corrida e da ausência de exclusão mútua. Os experimentos mostraram a importância do uso adequado de mutexes, semáforos e protocolos de sincronização para garantir consistência, integridade dos dados e funcionamento correto em aplicações concorrentes.

9. Referências e Fontes

- <https://github.com/codophobia/producer-consumer-problem-solution-in-c/tree/master>
- <https://github.com/stevellyall/producerconsumer3410>
- <https://www.geeksforgeeks.org/operating-systems/posix-threads-in-os/>
- <https://www.geeksforgeeks.org/c/thread-functions-in-c-c/>
- <https://github.com/alexandreantonaccio/SistemasOperacionais>
- Códigos gerados com auxílio de inteligência artificial